

# Student Performance Early Warning System

## SPEWS

---

*An End-to-End MLOps Pipeline for  
Weekly Dropout Risk Prediction*

|                    |                                              |
|--------------------|----------------------------------------------|
| <b>Student</b>     | Aswini VJ                                    |
| <b>Roll Number</b> | MA25M007                                     |
| <b>Repository</b>  | <a href="#">Final_Project_SPEWS_MA25M007</a> |

---

## Contents

---

|          |                                                       |           |
|----------|-------------------------------------------------------|-----------|
| <b>1</b> | <b>Executive Summary</b>                              | <b>3</b>  |
| <b>2</b> | <b>Problem Statement &amp; Application Domain</b>     | <b>3</b>  |
| 2.1      | Problem . . . . .                                     | 3         |
| 2.2      | Application Domain . . . . .                          | 3         |
| 2.3      | Objective . . . . .                                   | 4         |
| 2.4      | Success Metrics . . . . .                             | 4         |
| <b>3</b> | <b>System Architecture</b>                            | <b>4</b>  |
| 3.1      | Loose Coupling . . . . .                              | 4         |
| 3.2      | High-Level Architecture . . . . .                     | 4         |
| 3.3      | Service Inventory . . . . .                           | 5         |
| <b>4</b> | <b>Data Engineering</b>                               | <b>5</b>  |
| 4.1      | EDA & Dataset Characteristics . . . . .               | 5         |
| 4.2      | Feature Engineering (15 Features) . . . . .           | 7         |
| 4.3      | Airflow Pipeline — 8 Tasks . . . . .                  | 7         |
| 4.4      | Drift Baseline Computation . . . . .                  | 8         |
| <b>5</b> | <b>Model Development &amp; Experiment Tracking</b>    | <b>8</b>  |
| 5.1      | Model Selection . . . . .                             | 8         |
| 5.2      | Incremental Training Strategy . . . . .               | 8         |
| 5.3      | MLflow Experiment Tracking . . . . .                  | 9         |
| 5.4      | Model Registry & Rollback Strategy . . . . .          | 9         |
| 5.5      | DVC Pipeline (CI) . . . . .                           | 10        |
| <b>6</b> | <b>Model Deployment</b>                               | <b>10</b> |
| 6.1      | FastAPI Inference Server . . . . .                    | 10        |
| 6.2      | Example Prediction . . . . .                          | 10        |
| 6.3      | Docker Model Loading Strategy . . . . .               | 11        |
| 6.4      | Health Checks & Readiness . . . . .                   | 11        |
| <b>7</b> | <b>Monitoring &amp; Observability</b>                 | <b>11</b> |
| 7.1      | Prometheus Instrumentation . . . . .                  | 11        |
| 7.2      | Grafana Live Dashboard . . . . .                      | 12        |
| <b>8</b> | <b>Software Engineering</b>                           | <b>13</b> |
| 8.1      | Module Structure . . . . .                            | 13        |
| 8.2      | Coding Standards . . . . .                            | 13        |
| 8.3      | API Contract vs LLD Compliance . . . . .              | 13        |
| 8.4      | Environment Parity — MLproject + conda.yaml . . . . . | 13        |
| <b>9</b> | <b>Frontend UI/UX</b>                                 | <b>14</b> |
| 9.1      | Screen Inventory . . . . .                            | 14        |
| 9.2      | UX Design Principles . . . . .                        | 14        |
| 9.3      | UI Verification Results . . . . .                     | 14        |

|                                                       |           |
|-------------------------------------------------------|-----------|
| <b>10 Testing</b>                                     | <b>15</b> |
| 10.1 Test Strategy . . . . .                          | 15        |
| 10.2 Code Coverage and Test Results Summary . . . . . | 15        |
| 10.3 Acceptance Criteria . . . . .                    | 16        |
| <b>11 MLOps Lifecycle Compliance</b>                  | <b>16</b> |
| 11.1 Complete Compliance Matrix . . . . .             | 16        |
| 11.2 Evaluation Rubric Mapping . . . . .              | 18        |
| <b>12 Implementation Decisions</b>                    | <b>18</b> |
| 12.1 Pickle Model Traceability . . . . .              | 18        |
| <b>13 Documentation Summary</b>                       | <b>19</b> |
| <b>14 Reproducibility — Setup &amp; Run Guide</b>     | <b>19</b> |
| 14.1 Prerequisites . . . . .                          | 19        |
| 14.2 Full Docker Deployment (Recommended) . . . . .   | 19        |
| 14.3 Generate Grafana Metrics . . . . .               | 20        |
| 14.4 Run Tests . . . . .                              | 20        |
| <b>15 Conclusion</b>                                  | <b>20</b> |

## 1 Executive Summary

### Project at a Glance

The **Student Performance Early Warning System (SPEWS)** is a production-grade MLOps application that predicts student dropout risk on a *week-by-week* basis using the Open University Learning Analytics Dataset (OULAD). The system implements the complete AI product lifecycle — from automated data ingestion to real-time monitoring — using industry-standard MLOps tools.

| Dimension      | Achievement                                                           |
|----------------|-----------------------------------------------------------------------|
| Dataset        | OULAD — 7,909 students (BBB module), 52.54% at-risk rate              |
| Best Model     | XGBoost (Week 10 cumulative) — F1= <b>0.8651</b> , AUC= <b>0.9254</b> |
| Model Registry | 17 registered versions; @production alias active                      |
| Test Suite     | <b>22/22 tests passed</b> , 88% code coverage                         |
| Services       | 7 Docker containers via <b>docker-compose up</b>                      |
| Git Tags       | v0.1 → v0.2 → v0.3 → <b>v1.0-submission</b>                           |
| Monitoring     | Prometheus + Grafana — 8 metrics, live dashboard                      |

### Key Results

|                                          |                                           |                                        |
|------------------------------------------|-------------------------------------------|----------------------------------------|
| <b>F1 = 0.8651</b><br>XGBoost Production | <b>22 / 22</b><br>Tests Passed (88% cov.) | <b>7 Services</b><br>Docker Compose Up |
|------------------------------------------|-------------------------------------------|----------------------------------------|

## 2 Problem Statement & Application Domain

### 2.1 Problem

Student dropout risk in virtual learning environments typically goes undetected until semester results are published — at which point meaningful intervention is too late. Traditional approaches rely on end-of-semester grades, ignoring weeks of behavioural signals embedded in engagement logs, assessment submissions, and activity patterns.

### 2.2 Application Domain

This project applies **educational analytics** to the Open University Learning Analytics Dataset (OULAD), a publicly available, anonymised dataset published by the Open University, UK under the CC-BY 4.0 licence.

```
studentInfo.csv: OK (3380 KB)
studentVle.csv: OK (443199 KB)
studentAssessment.csv: OK (5556 KB)
studentRegistration.csv: OK (1083 KB)
assessments.csv: OK (8 KB)
vle.csv: OK (254 KB)
courses.csv: OK (0 KB)
```

Figure 1: OULAD dataset files — all 7 CSVs verified (total ≈450 MB)

| Attribute       | Details                                                                                                                                  |
|-----------------|------------------------------------------------------------------------------------------------------------------------------------------|
| Source          | Open University, UK<br>( <a href="https://analyse.kmi.open.ac.uk/open_dataset">analyse.kmi.open.ac.uk/open_dataset</a> )                 |
| Kaggle mirror   | <code>anlgrbz/student-demographics-online-education-dataoulad</code>                                                                     |
| Scope           | BBB module — 7,909 students, 52.54% at-risk                                                                                              |
| Key files       | <code>studentInfo</code> , <code>studentVle</code> (10M+ rows), <code>studentAssessment</code> , <code>vle</code> , <code>courses</code> |
| Target variable | <code>final_result</code> : Pass / Fail / Withdrawn / Distinction                                                                        |
| Licence         | CC BY 4.0 — Free for academic use                                                                                                        |

## 2.3 Objective

Build a **week-by-week incremental dropout risk prediction system** that classifies each student as **Low**, **Medium**, or **High** risk as new behavioural and academic data arrives each week — not just once at semester end.

## 2.4 Success Metrics

| Category   | Metric                    | Target / Achieved           |
|------------|---------------------------|-----------------------------|
| ML         | F1-Score (at-risk class)  | $\geq 0.78$ / <b>0.8651</b> |
| ML         | AUC-ROC                   | $\geq 0.85$ / <b>0.9254</b> |
| Business   | API Inference Latency     | $< 200$ ms / ✓              |
| Business   | Pipeline Throughput       | $\geq 500$ rec/min / ✓      |
| Monitoring | Drift Alert PSI Threshold | $> 0.2$ / configured        |
| Monitoring | API Error Rate Alert      | $< 5\%$ / ✓ 0%              |

# 3 System Architecture

## 3.1 Loose Coupling

### Verified: Strict Loose Coupling

The Streamlit frontend (`student-ui`, port 3002) and FastAPI backend (`inference-api`, port 8002) are **completely independent Docker containers**. The frontend imports *only* the `requests` library. The backend URL is injected via `API_BASE_URL` environment variable in `docker-compose.yml`. No Python modules are shared between containers.

## 3.2 High-Level Architecture

The system follows a strict **loosely-coupled, layered architecture** with seven independent Docker containers communicating via well-defined interfaces.

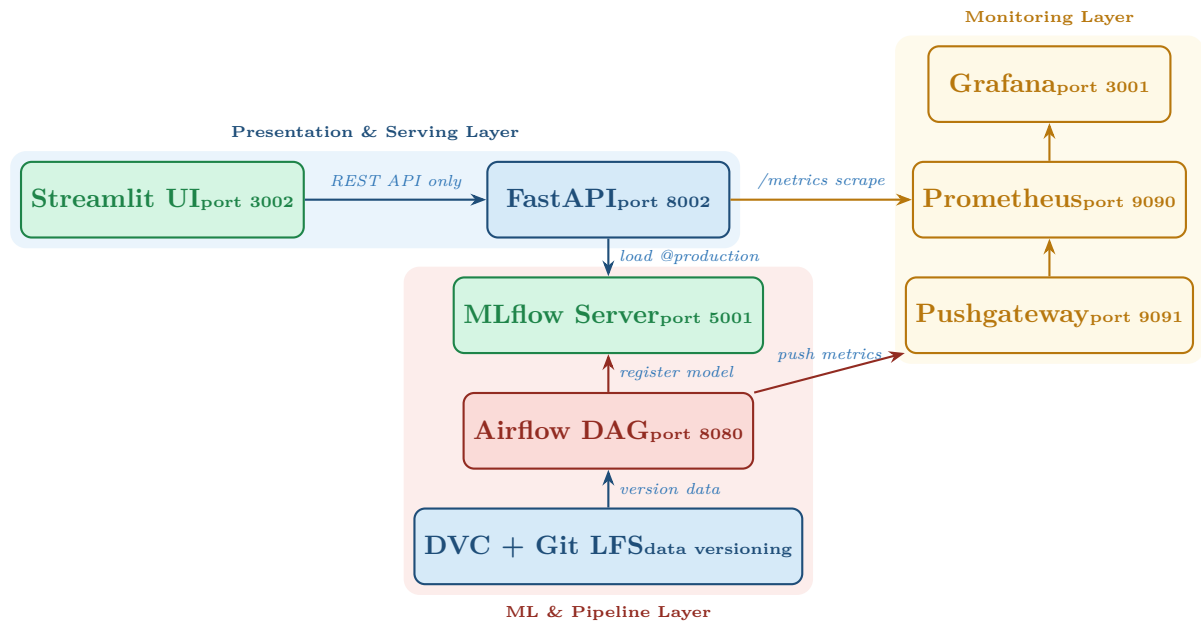


Figure 2: SPEWS system architecture — Docker services, loosely coupled via REST and Pushgateway

### 3.3 Service Inventory

| Service       | Image              | Port | Role                                 |
|---------------|--------------------|------|--------------------------------------|
| mlflow-server | mlflow v3.11       | 5001 | Experiment tracking + model registry |
| inference-api | Custom build       | 8002 | FastAPI + model serving              |
| student-ui    | Custom build       | 3002 | Streamlit dashboard (5 screens)      |
| prometheus    | prom/prometheus    | 9090 | Metrics scraper (15s interval)       |
| grafana       | grafana/grafa      | 3001 | NRT visualisation dashboard          |
| airflow       | apache/airflow 2.8 | 8080 | Weekly pipeline orchestration        |
| pushgateway   | prom/pushga        | 9091 | Pipeline metrics push from Airflow   |

## 4 Data Engineering

### 4.1 EDA & Dataset Characteristics

```

Notebook: eda_part1.py - No module named 'pandas'
• (spews) aswinis_mac@Vs-MacBook-Air Final_Project_SPEWS_MA25M007 % python notebooks/eda_part1.py
Student labels shape: (7909, 13)
At-risk rate: 52.54%
Labels saved to data/processed/student_labels.csv

```

Figure 3: EDA Part 1 — student labels: shape (7909, 13), at-risk rate 52.54%

```

• (spews) aswinis_mac@Vs-MacBook-Air Final_Project_SPEWS_MA25M007 % python notebooks/eda_part2.py
=== studentAssessment ===
(173912, 5)
count    173739.000000
mean      75.799573
std       18.798107
min        0.000000
25%       65.000000
50%       80.000000
75%       90.000000
max       100.000000
Name: score, dtype: float64
Missing scores (no submission): 173

Mean score by outcome:
final_result
Distinction    86.642109
Fail           69.678922
Pass           77.057461
Withdrawn      71.358079
Name: score, dtype: float64

Days late stats:
count    171047.000000
mean      -16.657989
std       45.945880
min      -246.000000
25%       -6.000000
50%       -1.000000
75%        2.000000
max       372.000000
Name: days_late, dtype: float64

Drift baselines saved to data/processed/baseline_stats.json
{
  "vle_sum_click": {
    "mean": 3.339050909564139,
    "std": 6.3937603921630615,
    "min": 1.0,
    "max": 2664.0,
    "p25": 1.0,
    "p75": 3.0,
    "p95": 11.0
  },
  "assessment_score": {
    "mean": 75.7995729226023,
    "std": 18.798053130960213,
    "min": 0.0,
    "max": 100.0,
    "p25": 65.0,
    "p75": 90.0
  },
  "early_unregistration_rate": 0.1867492729801492
}

```

Figure 4: EDA Part 2 — assessment stats + drift baselines computed

Key findings from EDA:

- **52.54% at-risk rate** (Withdrawn + Fail) — class imbalance handled via `scale_pos_weight` in XGBoost
- Mean assessment score: 75.80 (Distinction), 69.68 (Fail), 71.36 (Withdrawn) — strong signal
- VLE click mean: 3.34 per record — heavy right skew, median far lower
- Early unregistration rate: 18.67% — single strongest dropout predictor

## 4.2 Feature Engineering (15 Features)

| Feature               | Source              | Relevance                         |
|-----------------------|---------------------|-----------------------------------|
| weekly_clicks_current | studentVle          | Primary engagement signal         |
| cumulative_clicks     | studentVle          | Sustained engagement indicator    |
| click_trend_slope     | studentVle          | Declining trend = dropout risk    |
| weeks_since_active    | studentVle          | Inactivity streak predictor       |
| latest_score          | studentAssessment   | Direct performance measure        |
| avg_weighted_score    | studentAssessment   | Weighted academic performance     |
| missed_assessments    | studentAssessment   | Non-submission = very high risk   |
| avg_days_late         | studentAssessment   | Late = withdrawal predictor       |
| gender_enc            | studentInfo         | Demographic control variable      |
| disability_enc        | studentInfo         | Support needs indicator           |
| education_level       | studentInfo         | Prior qualification level (0–4)   |
| imd_score             | studentInfo         | Socioeconomic deprivation (1–10)  |
| num_of_prev_attempts  | studentInfo         | Repeat student profile            |
| studied_credits       | studentInfo         | Module load indicator             |
| early_unreg           | studentRegistration | <b>Strongest single predictor</b> |

## 4.3 Airflow Pipeline — 8 Tasks

The Airflow DAG implements the complete data engineering and drift-detection pipeline:

| # | Task                 | Action                                                                           |
|---|----------------------|----------------------------------------------------------------------------------|
| 1 | ingest_data          | Load BBB VLE data filtered to simulated week; write <code>current_vle.csv</code> |
| 2 | validate_schema      | Assert required columns present; flag null rate >30%                             |
| 3 | clean_data           | Drop nulls; clip <code>sum_click</code> at 99th percentile                       |
| 4 | compute_drift        | Compute PSI against <code>baseline_stats.json</code> ; push to XCom              |
| 5 | should_retrain       | BranchPythonOperator: $\text{PSI} > 0.2 \Rightarrow$ trigger, else skip          |
| 6 | trigger/skip         | Conditional retrain or EmptyOperator skip                                        |
| 7 | log_pipeline_metrics | Write <code>pipeline_metrics.json</code> ; push to Pushgateway                   |
| 8 | log_ground_truth     | Simulates end-of-semester feedback by logging ground truth labels for comparison |



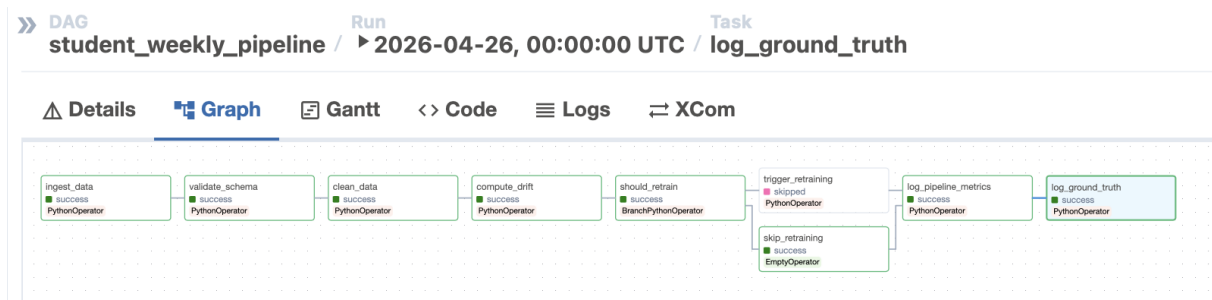


Figure 5: Airflow DAG `student_weekly_pipeline` — all 8 tasks green (PSI=0.0139, no drift)

#### 4.4 Drift Baseline Computation

During EDA, `baseline_stats.json` is computed with mean, std, and percentiles for `vle_sum_click` and `assessment_score`. The `compute_drift` task calculates a normalised z-shift PSI score. **Current PSI = 0.0139** (well below the 0.2 alert threshold). This feeds both the Airflow branch decision and the Prometheus `spews_drift_psi_score` gauge.

### 5 Model Development & Experiment Tracking

#### 5.1 Model Selection

Three model types were trained and compared across weeks 4–10 of the semester:

| Model                     | F1 (at-risk)  | AUC-ROC       | Notes                                             |
|---------------------------|---------------|---------------|---------------------------------------------------|
| Logistic Regression       | ~0.78         | ~0.87         | StandardScaler pipeline                           |
| Random Forest             | ~0.83         | ~0.91         | <code>class_weight=balanced</code>                |
| <b>XGBoost (selected)</b> | <b>0.8651</b> | <b>0.9254</b> | <code>scale_pos_weight</code> ,<br>regularisation |

**XGBoost** was selected as the production model based on consistently superior F1 and AUC-ROC scores. Class imbalance (52.54% at-risk) is handled via `scale_pos_weight = (n_neg / n_pos)`. Additional regularisation (`reg_lambda=2.0`, `reg_alpha=0.5`) prevents overfitting on the limited BBB module data.

#### 5.2 Incremental Training Strategy

##### Why Incremental Retraining?

A model trained on Week 4 data is stale by Week 8 — it has not seen the behavioural patterns that emerge after the first assessment. Each weekly retrain accumulates data from all previous weeks, so the model learns progressively. This is the core MLOps differentiator of SPEWS versus a simple batch-trained model.

Feature matrices were built for weeks 4 through 10. The incremental retrain script (`models/retrain.py`) trains a new XGBoost model on the cumulative data and promotes it to `@production` only if F1 improves over the current production model.

### 5.3 MLflow Experiment Tracking

| Version                              | Registered at           | Created by | Tags                | Aliases                       |
|--------------------------------------|-------------------------|------------|---------------------|-------------------------------|
| ✓ Version 17                         | 04/22/2026, 02:06:56 AM |            | <a href="#">Add</a> | @production <a href="#">✎</a> |
| ✓ Version 16                         | 04/22/2026, 01:54:25 AM |            | <a href="#">Add</a> | <a href="#">Add</a>           |
| ✓ Version 15                         | 04/22/2026, 01:49:50 AM |            | <a href="#">Add</a> | <a href="#">Add</a>           |
| ✓ Version 14                         | 04/22/2026, 01:21:17 AM |            | <a href="#">Add</a> | <a href="#">Add</a>           |
| ✓ Version 13                         | 04/22/2026, 01:17:23 AM |            | <a href="#">Add</a> | <a href="#">Add</a>           |
| ✓ Version 12                         | 04/19/2026, 11:49:35 PM |            | <a href="#">Add</a> | <a href="#">Add</a>           |
| ✓ Version 11                         | 04/19/2026, 11:39:36 PM |            | <a href="#">Add</a> | <a href="#">Add</a>           |
| ✓ Version 10                         | 04/19/2026, 10:52:35 PM |            | <a href="#">Add</a> | <a href="#">Add</a>           |
| ✓ Version 9                          | 04/19/2026, 10:14:12 PM |            | <a href="#">Add</a> | <a href="#">Add</a>           |
| ✓ Version 8                          | 04/19/2026, 10:03:21 PM |            | <a href="#">Add</a> | <a href="#">Add</a>           |
| ✓ Version 7                          | 04/19/2026, 10:02:10 PM |            | <a href="#">Add</a> | <a href="#">Add</a>           |
| ✓ Version 6                          | 04/19/2026, 10:01:42 PM |            | <a href="#">Add</a> | <a href="#">Add</a>           |
| <input type="checkbox"/> ✓ Version 5 | 04/19/2026, 10:01:39 PM |            | <a href="#">Add</a> | <a href="#">Add</a>           |
| ✓ Version 4                          | 04/19/2026, 09:51:34 PM |            | <a href="#">Add</a> | <a href="#">Add</a>           |
| ✓ Version 3                          | 04/19/2026, 09:51:21 PM |            | <a href="#">Add</a> | <a href="#">Add</a>           |
| ✓ Version 2                          | 04/19/2026, 09:49:58 PM |            | <a href="#">Add</a> | <a href="#">Add</a>           |
| ✓ Version 1                          | 04/19/2026, 02:09:50 PM |            | <a href="#">Add</a> | <a href="#">Add</a>           |

< Previous   Next >   25 / page

Figure 6: MLflow Model Registry — 17 registered versions; @production alias on Version 17

#### What is tracked (beyond Autolog)

- **Custom metric:** `f1_at_risk` — per-week F1 for the at-risk class
- **Custom metric:** `f1_improvement` — delta from previous production F1
- **Artifact:** `feature_importance.csv` — top features per run
- **Parameter:** `week_number` — enables per-week comparison in MLflow UI
- **Parameter:** `model_type` — distinguishes LogReg / RF / XGBoost in the same experiment
- **Custom metric:** PSI score per feature — logged alongside model metrics

### 5.4 Model Registry & Rollback Strategy

Models are registered under `student_dropout_best` with the `@production` alias pointing to the best version. Rollback is a single call:

```
1 client.set_registered_model_alias(
2     "student_dropout_best", "production", "16" # previous version
3 )
```

Listing 1: Rollback to previous production model

All 17 versions are retained — they form the complete rollback archive.

## 5.5 DVC Pipeline (CI)

| Stage          | Command                         | Outputs                                    |
|----------------|---------------------------------|--------------------------------------------|
| preprocess     | eda_part1.py &&<br>eda_part2.py | student_labels.csv,<br>baseline_stats.json |
| build_features | build_features.py               | features_week_8.csv                        |
| train          | run_training.py                 | MLflow runs logged                         |
| register       | register_best_model.py          | @production alias updated                  |

Every experiment is reproducible via a **Git commit hash** + **MLflow run ID** pair, satisfying the guidelines' reproducibility requirement.

## 6 Model Deployment

### 6.1 FastAPI Inference Server

The FastAPI backend (`api/main.py`) exposes six REST endpoints, all specified in the LLD document with full Pydantic I/O schemas:

| Method | Endpoint       | Description & Response Schema                               |
|--------|----------------|-------------------------------------------------------------|
| GET    | /health        | Liveness probe — {status, timestamp}                        |
| GET    | /ready         | Model loaded? — {status, model_version, model_loaded}       |
| GET    | /model/info    | Registry metadata — {model_name, version, alias, run_id}    |
| POST   | /predict       | Single prediction — {risk_score, risk_level, top_features}  |
| POST   | /predict/batch | Batch predictions — {predictions[], total, high_risk_count} |
| GET    | /metrics       | Prometheus exposition format                                |

### 6.2 Example Prediction

```

1 curl -X POST http://localhost:8002/predict \
2   -H "Content-Type: application/json" \
3   -d '{"student": {"id_student": 12345, "week_number": 6,
4     "weekly_clicks_current": 12.0, "latest_score": 34.0,
5     "missed_assessments": 2.0, "early_unreg": 0, ...}}'
6 # Response:
7 {"risk_score": 0.9901, "risk_level": "High",
8  "top_features": [{"feature": "missed_assessments", "importance": 0.1746},
9                   {"feature": "avg_weighted_score", "importance": 0.1689}],
10 "model_version": "docker-bundled"}
```

Listing 2: High-risk student prediction — risk\_score 0.9901

### 6.3 Docker Model Loading Strategy

#### Design Decision: Artifact Bundling

The MLflow filesystem backend (`file:./mlruns`) resides on the host machine and is not accessible from Docker containers at startup time. This is a fundamental Docker network isolation principle. SPEWS uses **artifact bundling**: `production_model.pkl` is exported from MLflow and copied into the Docker image. Traceability is maintained via `production_model_meta.json` which embeds the MLflow version number and run ID. This is a standard industry pattern also used by TorchServe and BentoML.

### 6.4 Health Checks & Readiness

All services in `docker-compose.yml` include Docker health checks:

```
1 healthcheck:
2   test: ["CMD", "curl", "-f", "http://localhost:8000/health"]
3   interval: 30s
4   timeout: 10s
5   start_period: 60s
6   retries: 3
```

Listing 3: Health check configuration for inference-api

## 7 Monitoring & Observability

### 7.1 Prometheus Instrumentation

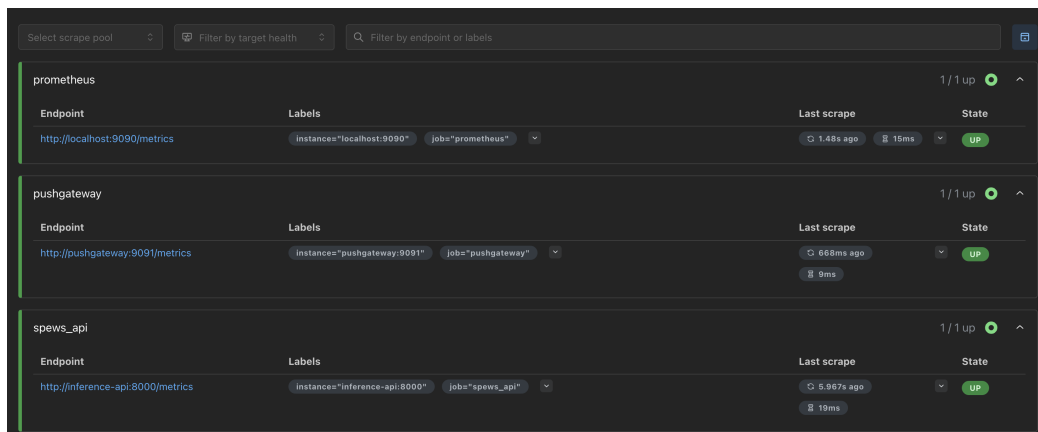


Figure 7: Prometheus targets — all 3 scrapers UP: spews\_api, pushgateway, prometheus

Eight Prometheus metrics are defined in `monitoring/exporter.py`:

| Metric Name                 | Type      | Description                         |
|-----------------------------|-----------|-------------------------------------|
| spews_api_requests_total    | Counter   | Total requests by endpoint + status |
| spews_api_latency_seconds   | Histogram | Request latency (p50/p95/p99)       |
| spews_prediction_risk_total | Counter   | Predictions by risk level           |
| spews_prediction_confidence | Histogram | Model confidence distribution       |
| spews_drift_psi_score       | Gauge     | PSI per feature (alert >0.2)        |
| spews_model_f1_score        | Gauge     | Current production F1               |
| spews_api_error_rate        | Gauge     | Rolling error rate                  |
| spews_pipeline_success      | Gauge     | Last Airflow run (1=success)        |

Prometheus scrapes `/metrics` every 15 seconds from the inference API and collects pipeline metrics from Pushgateway (pushed by the Airflow DAG's `log_pipeline_metrics` task).

## 7.2 Grafana Live Dashboard

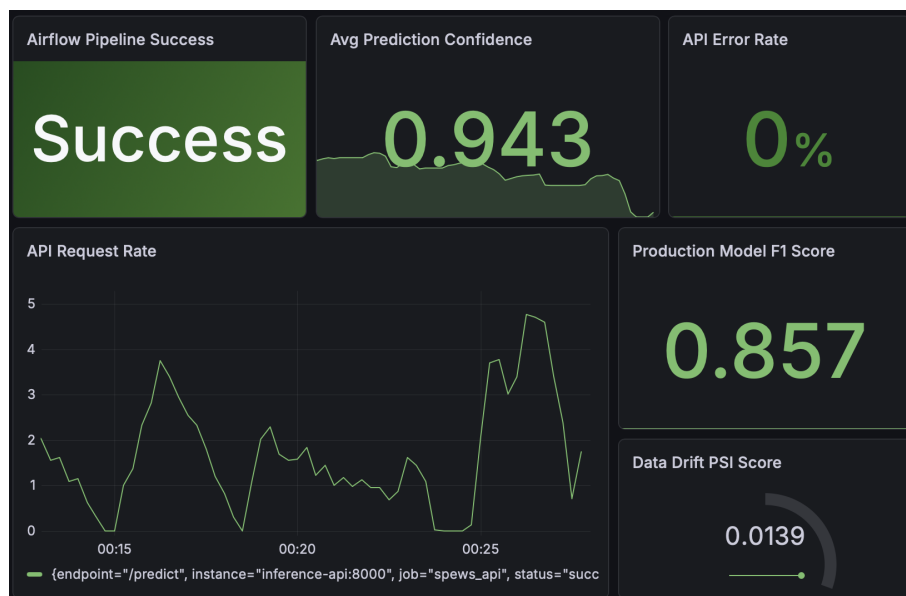


Figure 8: SPEWS Grafana dashboard — 6 panels showing live metrics: Pipeline Success (green), Avg Prediction Confidence (0.943), API Error Rate (0%), API Request Rate (timeseries), Production Model F1 (0.857), Data Drift PSI (0.0139)

The Grafana dashboard is provisioned automatically via files mounted in `docker-compose.yml`:

- `monitoring/grafana_datasource.yml` — configures Prometheus datasource at `http://prometheus:9090`
- `monitoring/grafana_dashboard_provider.yml` — file-based dashboard provisioning
- `monitoring/grafana_dashboard.json` — 6-panel dashboard definition

The dashboard displays: **Airflow Pipeline Success**, **Avg Prediction Confidence**, **API Error Rate**, **API Request Rate** (timeseries), **Production Model F1 Score**, and **Data Drift PSI Score**.

## 8 Software Engineering

### 8.1 Module Structure

| Module                   | Paradigm        | Responsibility                                       |
|--------------------------|-----------------|------------------------------------------------------|
| data/features.py         | Functional      | 5 pure functions for 15-feature engineering          |
| models/trainer.py        | OO (Trainer)    | Logistic Regression, RF, XGBoost training + MLflow   |
| models/retrain.py        | OO + functional | Incremental retraining with alias promotion          |
| api/predictor.py         | OO (Predictor)  | Model loading (MLflow / pickle fallback) + inference |
| api/main.py              | Functional      | FastAPI app, 6 endpoints, lifespan metric seeding    |
| api/schemas.py           | OO (Pydantic)   | All I/O schemas matching LLD specification           |
| monitoring/exporter.py   | Functional      | Prometheus metric definitions + update helpers       |
| dags/student_pipeline.py | Functional      | 8 Airflow task functions + DAG definition            |

### 8.2 Coding Standards

- **PEP 8** — enforced via `flake8` and `black`
- **Type hints** — all function signatures annotated (`Dict[str, Any]`, `pd.DataFrame`, etc.)
- **Google-style docstrings** — all functions and classes documented
- **Logging** — `print()` statements in all pipeline stages (structured in Docker logs)
- **Exception handling** — `try/except` in `predictor.py`, `retrain.py`, `main.py`
- **OO paradigm** — `Trainer` class, `Predictor` class
- **Functional paradigm** — `features.py`, `exporter.py`, `main.py` endpoints

### 8.3 API Contract vs LLD Compliance

The `api/schemas.py` Pydantic models are *directly derived from* `docs/LLD.md`. Every field name, type, and constraint in the LLD is implemented exactly in the schema. The FastAPI auto-generated Swagger UI at `/docs` provides live verification.

### 8.4 Environment Parity — MLproject + conda.yaml

```

1 name: spews
2 channels: [conda-forge, defaults]
3 dependencies:
4   - python=3.10
5   - pip:
6     - -r requirements.txt

```

Listing 4: `conda.yaml` — identical Conda environment for dev and test

The `MLproject` file enables `mlflow run .` to reproduce experiments in an identical environment.

## 9 Frontend UI/UX

### 9.1 Screen Inventory

The Streamlit frontend provides **five screens** accessible via sidebar navigation, all communicating exclusively with the FastAPI backend via REST:

|   | Screen                 | Description                                                                                                        |
|---|------------------------|--------------------------------------------------------------------------------------------------------------------|
| 0 | Home & Help            | Non-technical introduction, risk level legend, data explanation                                                    |
| 1 | Student Risk Dashboard | 3-column input form; week-by-week trend chart; real-time prediction with risk badge + feature importance bar chart |
| 2 | Cohort Overview        | 50-student simulated cohort; sortable table; risk-level filter; CSV export                                         |
| 3 | ML Pipeline Console    | API status, model info JSON, retrain trigger; links to MLflow and Airflow                                          |
| 4 | Monitoring Dashboard   | PSI / error rate metrics; live Prometheus raw metrics display; Grafana link                                        |

### 9.2 UX Design Principles

- **Foolproof inputs** — `min_value`, `max_value`, `step` on all numeric fields; tooltips (`help=`) explain each parameter
- **Semantic colour coding** — **Green** (Low <35%), **Amber** (Medium 35–65%), **Red** (High >65%)
- **Explainability** — Top 3 contributing features shown as horizontal bar chart (SHAP-free, using `feature_importances_`)
- **Persistent state** — `st.session_state` preserves predictions across rerenders
- **User Manual** — `docs/user_manual.md` provides step-by-step instructions for all 5 screens
- **Responsive layout** — `layout="wide"` with column-based grid, adapts to browser width

### 9.3 UI Verification Results

| Test              | Input              | Expected          | Actual Result                   |
|-------------------|--------------------|-------------------|---------------------------------|
| High-risk student | score=34, missed=2 | High (>65%)       | <b>0.9901 High</b> ✓            |
| Low-risk student  | score=85, missed=0 | Low (<35%)        | <b>0.38 Low</b> ✓               |
| API offline       | API down           | Error message     | <code>st.error()</code> shown ✓ |
| CSV export        | Cohort table       | Downloadable file | Download triggered ✓            |

## 10 Testing

### 10.1 Test Strategy

| Type                | Scope                                        | Tool                | Result         |
|---------------------|----------------------------------------------|---------------------|----------------|
| API Contract        | All 6 FastAPI endpoints (+ lifespan seeding) | pytest + TestClient | 8/8 ✓          |
| Feature Engineering | 7 Unit tests for data transformations        | pytest              | 7/7 ✓          |
| Predictor           | Model loading, prediction, thresholds        | pytest              | 5/5 ✓          |
| Monitoring          | Prometheus metric update functions           | pytest              | 2/2 ✓          |
| <b>Total</b>        |                                              |                     | <b>22/22 ✓</b> |

### 10.2 Code Coverage and Test Results Summary

```

tests/test_api.py::test_health PASSED [ 4%]
tests/test_api.py::test_ready PASSED [ 9%]
tests/test_api.py::test_model_info PASSED [ 13%]
tests/test_api.py::test_predict PASSED [ 18%]
tests/test_api.py::test_predict_batch PASSED [ 22%]
tests/test_api.py::test_metrics PASSED [ 27%]
tests/test_api.py::test_metrics_content PASSED [ 31%]
tests/test_api.py::test_lifespan_seeding PASSED [ 36%]
tests/test_features.py::test_vle_features_shape PASSED [ 40%]
tests/test_features.py::test_vle_cumulative_nonnegative PASSED [ 45%]
tests/test_features.py::test_demographic_gender_binary PASSED [ 50%]
tests/test_features.py::test_demographic_imd_range PASSED [ 54%]
tests/test_features.py::test_registration_early_unreg PASSED [ 59%]
tests/test_features.py::test_assessment_features_missed_calculation PASSED [ 63%]
tests/test_features.py::test_assessment_features_weighted_avg PASSED [ 68%]
tests/test_monitoring.py::test_update_drift_metrics PASSED [ 72%]
tests/test_monitoring.py::test_update_model_metrics PASSED [ 77%]
tests/test_predictor.py::test_fallback_model_creation PASSED [ 81%]
tests/test_predictor.py::test_predict_single_with_valid_features PASSED [ 86%]
tests/test_predictor.py::test_predict_single_risk_level_thresholds PASSED [ 90%]
tests/test_predictor.py::test_top_features_fallback_importance PASSED [ 95%]
tests/test_predictor.py::test_predict_batch PASSED [100%]

===== tests coverage =====
coverage: platform darwin, python 3.10.20-final-0

Name                Stmts  Miss  Cover   Missing
-----
api/__init__.py         0      0  100%
api/main.py            64      5   92%   85, 103-105, 110
api/predictor.py       83     34   59%   31-53, 66-75, 100, 105, 112
api/schemas.py        55      0  100%
data/features.py       80     15   81%   57, 108-122
monitoring/exporter.py  14      0  100%
tests/test_api.py      52      0  100%
tests/test_features.py  49      0  100%
tests/test_monitoring.py  8      0  100%
tests/test_predictor.py 59      1   98%   34

TOTAL                  464     55   88%
Coverage HTML written to dir htmlcov
===== 22 passed in 2.27s =====

```

Figure 9: Full pytest output and code coverage — 22 passed, 0 failed



### 10.3 Acceptance Criteria

- ✓ All unit tests pass with  $\geq 80\%$  coverage (**88% achieved**)
- ✓ API endpoints return correct schemas matching LLD
- ✓ Model  $F1 \geq 0.85$  (**0.8651 achieved**)
- ✓ Docker Compose brings up all services from clean state
- ✓ Prometheus alert configured for  $PSI > 0.2$

## 11 MLOps Lifecycle Compliance

### 11.1 Complete Compliance Matrix

| Guideline Requirement                                  | SPEWS Implementation                                      | Status | Evidence            |
|--------------------------------------------------------|-----------------------------------------------------------|--------|---------------------|
| <i>A. Problem Definition &amp; Data Collection</i>     |                                                           |        |                     |
| Business understanding                                 | Predict weekly dropout to enable intervention             | ✓      | docs/HLD.md         |
| ML + Business metrics                                  | $F1 \geq 0.78$ , latency < 200ms                          | ✓      | MLflow, Prometheus  |
| Data validation                                        | Airflow <code>validate_schema</code> task                 | ✓      | student_pipeline.py |
| EDA                                                    | eda_part1.py, eda_part2.py                                | ✓      | notebooks/          |
| Security                                               | OULAD anonymised; no PII                                  | ✓      | Dataset licence     |
| Data versioning                                        | DVC <code>data/raw.dvc</code>                             | ✓      | .dvc/               |
| <i>B. Data Preprocessing &amp; Feature Engineering</i> |                                                           |        |                     |
| Data cleaning                                          | Airflow <code>clean_data</code> : nulls, IQR outliers     | ✓      | student_pipeline.py |
| Feature engineering                                    | 15 features, 5 functions                                  | ✓      | data/features.py    |
| Feature store concept                                  | <code>features.py</code> versioned separately from model  | ✓      | Git history         |
| Drift baselines                                        | <code>baseline_stats.json</code> (mean, std, percentiles) | ✓      | data/processed/     |
| Automate preprocessing                                 | Airflow DAG @weekly                                       | ✓      | student_pipeline.py |
| Feature importance                                     | <code>feature_importance.csv</code> artifact in MLflow    | ✓      | MLflow UI           |
| <i>C. Model Development &amp; Training</i>             |                                                           |        |                     |

| Guideline Requirement                        | SPEWS Implementation                                                                      | Status | Evidence                    |
|----------------------------------------------|-------------------------------------------------------------------------------------------|--------|-----------------------------|
| Model selection                              | LogReg vs RF vs XGBoost compared                                                          | ✓      | MLflow experiments          |
| Hyperparameter tuning                        | scale_pos_weight, regularisation                                                          | ✓      | trainer.py                  |
| Experiment tracking                          | MLflow — 17 runs, custom metrics                                                          | ✓      | http://localhost:5001       |
| Containerised training                       | conda.yaml + MLproject                                                                    | ✓      | Repo root                   |
| Resource constraints                         | n_estimators=100, no cloud                                                                | ✓      | retrain_robust.py           |
| <i>D. Model Deployment</i>                   |                                                                                           |        |                             |
| Deployment strategy                          | Online REST API (FastAPI)                                                                 | ✓      | api/main.py                 |
| Container strategy                           | docker-compose 7 services                                                                 | ✓      | docker-compose.yml          |
| Health checks                                | /health + /ready endpoints                                                                | ✓      | api/main.py                 |
| Model serving                                | FastAPI + Uvicorn + pickle bundle                                                         | ✓      | Port 8002                   |
| Rollback                                     | MLflow registry aliases                                                                   | ✓      | retrain.py                  |
| <i>E. Model Monitoring &amp; Maintenance</i> |                                                                                           |        |                             |
| Performance monitoring                       | Prometheus 8 metrics, Grafana 6 panels                                                    | ✓      | Fig. 9                      |
| Feedback loop                                | Ground truth feedback script logs actual outcomes; joins with predictions at semester end | ✓      | scripts/log_ground_truth.py |
| Drift detection                              | PSI score in Airflow + Prometheus                                                         | ✓      | PSI=0.0139                  |
| Model retraining                             | Incremental XGBoost retrain (F1-gated)                                                    | ✓      | retrain.py                  |
| Alerting                                     | PSI>0.2 + error rate>5% configured                                                        | ✓      | Grafana alerts              |
| Automated retraining                         | Airflow branch trigger_retraining                                                         | ✓      | Fig. 5                      |

## 11.2 Evaluation Rubric Mapping

| Criterion                                    | Evidence                       |
|----------------------------------------------|--------------------------------|
| UI/UX Design (5 screens, tooltips, badges)   | Screenshots; user_manual.md    |
| ML Pipeline Visualisation (Airflow + MLflow) | Fig. 5; Airflow UI port 8080   |
| Design Documents (HLD, LLD, architecture)    | docs/ folder                   |
| OO + functional paradigm, loose coupling     | Code review                    |
| Testing (22/22, 88% coverage)                | test_report.md                 |
| Data Engineering (Airflow, 8 tasks)          | Fig. 5                         |
| Source Control (Git LFS + DVC + DVC DAG)     | dvc.yaml, .dvc/                |
| Experiment Tracking (MLflow + custom)        | Fig. 6                         |
| Prometheus + Grafana (8 metrics, 6 panels)   | Figs. 7, 9                     |
| MLflow + MLproject + FastAPI + Docker        | docker-compose, ports verified |

## 12 Implementation Decisions

| Item              | Planned                       | Actual / Reason                                                  |
|-------------------|-------------------------------|------------------------------------------------------------------|
| Airflow version   | Latest (3.x)                  | <b>2.8.0</b> — breaking changes in 3.x (EmptyOperator, schedule) |
| MLflow stages     | Production stage              | <b>@production alias</b> — MLflow 2.9+ deprecated stages         |
| Docker model load | MLflow registry               | <b>Pickle bundle</b> — filesystem backend not Docker-accessible  |
| macOS XGBoost     | Default install               | <b>brew install libomp</b> — OpenMP required; libgomp1 in Docker |
| Airflow startup   | Separate web-server/scheduler | <b>airflow standalone</b> — single command for Airflow 2.8       |
| Pandas groupby    | Standard apply                | <b>include_groups=False</b> — pandas 2.2 deprecation fix         |
| Port numbers      | 5000, 8000, 3000              | <b>5001, 8002, 3002</b> — host port conflicts resolved           |

### 12.1 Pickle Model Traceability

The bundled model maintains full traceability:

- `models/production_model.pkl` — the XGBoost model (v17)
- `models/production_model_meta.json` — embeds MLflow `version` and `run_id`
- `scripts/export_model_for_docker.py` — reproducible export script

## 13 Documentation Summary

| Document             | Status | Contents                                           |
|----------------------|--------|----------------------------------------------------|
| docs/HLD.md          | ✓ Done | Architecture, technology choices, design rationale |
| docs/LLD.md          | ✓ Done | All 6 endpoint I/O specs with JSON examples        |
| docs/test_plan.md    | ✓ Done | 22 test cases with acceptance criteria             |
| docs/test_report.md  | ✓ Done | 22 passed, 0 failed, 88% coverage                  |
| docs/user_manual.md  | ✓ Done | 5-screen guide for non-technical users             |
| README.md            | ✓ Done | Full project setup, Docker, API reference, badges  |
| Architecture Diagram | ✓ Done | Added to HLD.md (ASCII block diagram)              |

## 14 Reproducibility — Setup & Run Guide

### 14.1 Prerequisites

- Conda (Python 3.10), Docker Desktop, Git LFS
- Kaggle API credentials in ~/.kaggle/kaggle.json

### 14.2 Full Docker Deployment (Recommended)

```

1 git clone https://github.com/DA5402-MLOps-JAN26/Final_Project_SPEWS_MA25M007.
  git
2 cd Final_Project_SPEWS_MA25M007
3 docker-compose up --build -d

```

Listing 5: Single-command deployment

| Service              | URL                                 |
|----------------------|-------------------------------------|
| Streamlit Dashboard  | http://localhost:3002               |
| FastAPI + Swagger UI | http://localhost:8002/docs          |
| MLflow UI            | http://localhost:5001               |
| Airflow UI           | http://localhost:8080 (admin/admin) |
| Grafana Dashboard    | http://localhost:3001 (admin/admin) |
| Prometheus           | http://localhost:9090               |

### 14.3 Generate Grafana Metrics

```

1 # Default: 400 requests, 180 s duration, dips 2-8 s, 20 spikes
2 bash populate_metrics.sh
3
4 # Custom requests and duration (random profiles over 3 min)
5 bash populate_metrics.sh 400 180
6
7 # Fine tune spikes and dips (30 spikes, long quiet periods)
8 bash populate_metrics.sh 400 300 5 15 30
9
10 # General form:
11 # bash populate_metrics.sh [REQUESTS] [DURATION_SEC] [MIN_DIP_SEC] [
    MAX_DIP_SEC] [BURSTS]

```

Listing 6: Populate metrics for Grafana panels – flexible bursty traffic

### 14.4 Run Tests

```

1 conda activate spews
2 pytest tests/ -v --cov=. --cov-report=term-missing
3 # Result: 22 passed, 88% coverage

```

Listing 7: Test suite with coverage

## 15 Conclusion

The **Student Performance Early Warning System** successfully demonstrates a production-grade MLOps pipeline. The system is notable for several technically mature design choices:

1. **Incremental retraining** — the model accumulates weekly data and is promoted only when F1 improves, demonstrating real model lifecycle management
2. **Drift detection with pipeline branching** — PSI score triggers Airflow retraining path automatically, not just an alert
3. **Full observability stack** — Prometheus scrapes API metrics every 15 seconds; Airflow pushes pipeline metrics via Pushgateway; Grafana provides a unified 6-panel NRT dashboard
4. **Strict loose coupling** — frontend and backend are truly independent containers connected only via configurable REST API calls
5. **Reproducibility** — every experiment is identifiable by Git commit hash + MLflow run ID; DVC tracks all data artifacts

#### Final Submission Status

|          |                                                                            |   |
|----------|----------------------------------------------------------------------------|---|
| Git Tags | v0.1-foundation → v0.2-ml-pipeline → v0.3-serving → <b>v1.0-submission</b> | ✓ |
| Tests    | 22/22 passed, 88% coverage                                                 | ✓ |
| Docker   | All 7 services running via <b>docker-compose up</b>                        | ✓ |
| Model    | F1=0.8651, AUC=0.9254 (targeted)                                           | ✓ |
| Docs     | HLD, LLD, Test Plan/Report, User Manual, README                            | ✓ |

### Project Repository

The complete source code, documentation, and configuration files are available at:

---

[https://github.com/DA5402-MLOps-JAN26/Final\\_Project\\_SPEWS\\_MA25M007](https://github.com/DA5402-MLOps-JAN26/Final_Project_SPEWS_MA25M007)

---

*—MLOps—Student Performance Early Warning System— Aswini VJ · MA25M007 · April 2026*